



Introduction to NoSQL Databases

MongoDB, DynamoDB, and Modern Data Modeling

Chapter Overview

- 01** NoSQL Fundamentals and Data Modeling
- 02** MongoDB Architecture and CRUD
- 03** Indexes and Aggregation Pipelines
- 04** DynamoDB Core Concepts
- 05** Secondary Indexes and DynamoDB Streams
- 06** Choosing the Right Database



NoSQL Fundamentals

Section 1



Relational vs. NoSQL Trade-Offs

Relational Strengths

- ACID transactions, strong consistency, mature tooling
- Supports partitioning, replication, and online schema changes at scale

Where NoSQL Fills Gaps

- Complex JOINS across tables become expensive at very large scale
- Rigid schemas slow down rapid iteration on evolving data models
- Horizontal scaling across commodity hardware is a first-class feature



The CAP Theorem in Practice

Three Properties

- Consistency — every read returns the most recent write
- Availability — every request receives a response
- Partition Tolerance — system works despite network failures

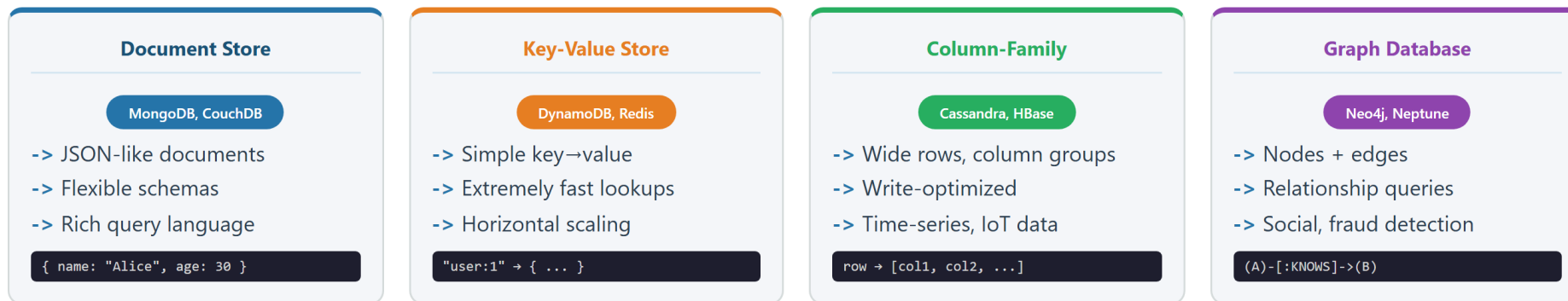
The Real Trade-Off

- Network partitions will happen in distributed systems — they are not optional
- Under a partition, the system must choose between consistency and availability
- MongoDB replica sets typically read from the primary by default, but actual consistency and availability behavior depends on read preference, read concern, and write concern
- DynamoDB offers eventually consistent reads by default, with strongly consistent reads supported on tables and LSIs only



NoSQL Database Categories

NoSQL Database Categories



Choose the model that matches your data relationships and access patterns

Schema Design Thinking

Denormalization

- In NoSQL, duplicating data is often preferred over JOINS
- Design for how you query, not how you store

Embedding vs. Referencing

- Embed related data when it is always accessed together (e.g., order + line items)
- Reference by ID when data is shared across documents or very large

Know Your Access Patterns First

- Model your schema around the queries your application will run most often



Relational vs. Document Model

Relational vs. Document Data Model

Relational (Normalized)

users

id	name	email
1	Alice	alice@co.com

orders (FK → users.id)

id	user_id	item	total
10	1	Widget	29.99

- > Data split across tables
- > JOINS required for reads
- > Strong consistency via ACID



Document (Embedded)

```
// users collection
{
  "_id": ObjectId("..."),
  "name": "Alice",
  "email": "alice@co.com",
  "orders": [
    { "item": "Widget",
      "total": 29.99 }
  ]
}
```

- > Related data in one document
- > Single read fetches everything
- > Flexible, evolving schema

Design for your query patterns — embed when data is read together, reference when shared

MongoDB Architecture and CRUD

Section 2



MongoDB Core Concepts

Data Hierarchy

- Database → Collections → Documents (analogous to DB → Tables → Rows)
- Documents are BSON (Binary JSON) — supports Date, Decimal128, ObjectId, etc.

The _id Field

- Every document must have a unique _id — MongoDB auto-generates an ObjectId if omitted
- You can supply your own _id (string, number, UUID) when inserting

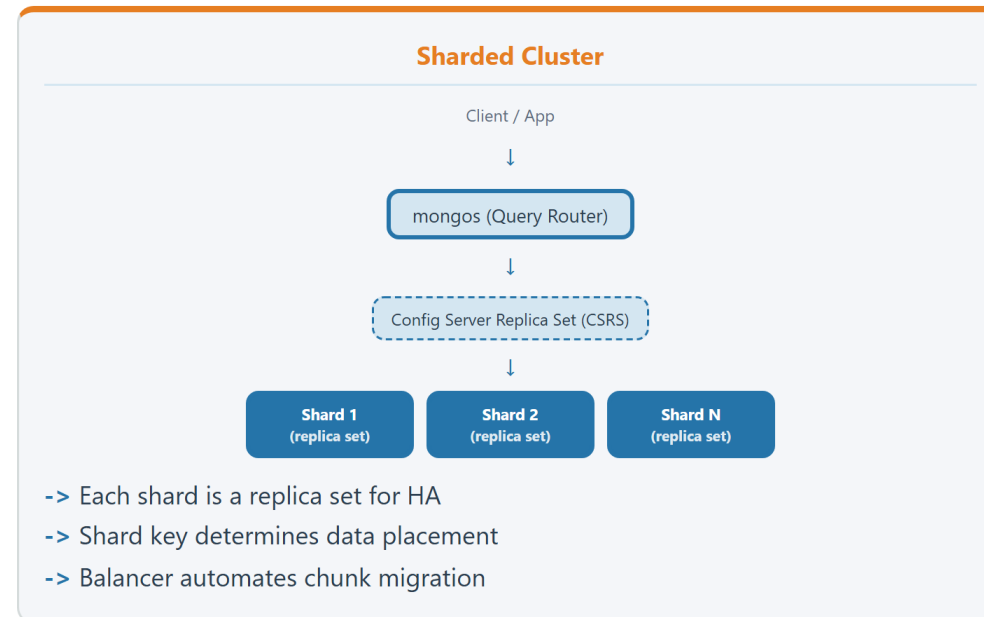
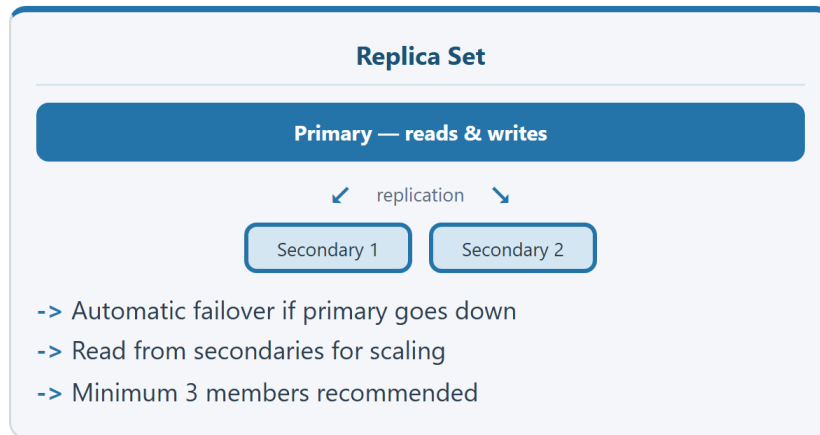
Flexible Schema

- Documents in the same collection can have different fields
- Schema validation rules can be added when you need enforcement



MongoDB Scaling Architecture

MongoDB Scaling Architecture



Replica sets provide high availability; sharding provides horizontal scalability

Replica Sets and High Availability

How Replication Works

- One Primary receives all writes; Secondaries replicate asynchronously
- If the Primary fails, an election promotes a Secondary automatically

Read Preferences

- primary — all reads from Primary (default, strongest consistency)
- secondaryPreferred — reads from Secondaries when available

Sharding for Horizontal Scale

- Data partitioned by shard key across multiple replica sets
- mongos router directs queries to the correct shard(s)



MongoDB CRUD Operations

```
// Insert a document
db.users.insertOne({ name: "Alice", email: "alice@test.com", age: 30 })

// Find with compound filter
db.users.find({ age: { $gte: 25 }, email: { $regex: /@test\.com$/ } })

// Update by unique field – safer than matching by name
db.users.updateOne(
  { email: "alice@test.com" },
  { $set: { age: 31 }, $push: { tags: "verified" } }
)

// Upsert – insert if no match, update if found
db.users.updateOne(
  { email: "bob@test.com" },
  { $set: { name: "Bob", age: 28 } },
  { upsert: true }
)

// Delete by unique identifier
db.users.deleteOne({ email: "alice@test.com" })
```

Querying and Filtering

Comparison Operators

- \$eq, \$ne, \$gt, \$gte, \$lt, \$lte — compare field values
- \$in, \$nin — match against an array of candidate values

Logical Operators

- \$and, \$or, \$not, \$nor — combine multiple filter conditions

Projections and Sorting (Shell Syntax)

- Select fields: `db.users.find({}, { name: 1, email: 1 })` — 1 = include, 0 = exclude
- Sort results: `db.users.find().sort({ age: -1 }).limit(10)`



Advanced Query Patterns

```
// Compound filter with $and (implicit) and $or
db.orders.find({
  status: "active",
  $or: [
    { total: { $gte: 100 } },
    { priority: "high" }
  ]
})

// Array queries
db.products.find({ tags: { $all: ["sale", "electronics"] } })

// Nested document query
db.users.find({ "address.city": "Seattle" })

// Basic text search (requires text index)
// For production full-text search, prefer MongoDB Search / Atlas Search
db.articles.find({ $text: { $search: "mongodb tutorial" } })
```



Indexes and Aggregation Pipelines

Section 3



MongoDB Indexes

Why Indexes Matter

- Without an index, MongoDB scans every document (collection scan) — $O(n)$
- Indexes reduce query time to $O(\log n)$ using B-tree structures

Index Types

- Single-field — index on one field, e.g., { email: 1 }
- Compound — multiple fields, e.g., { status: 1, createdAt: -1 }
- Multikey — automatically indexes each element in an array field
- Text indexes support basic \$text queries; for production full-text search, prefer MongoDB Search / Atlas Search



Creating and Using Indexes

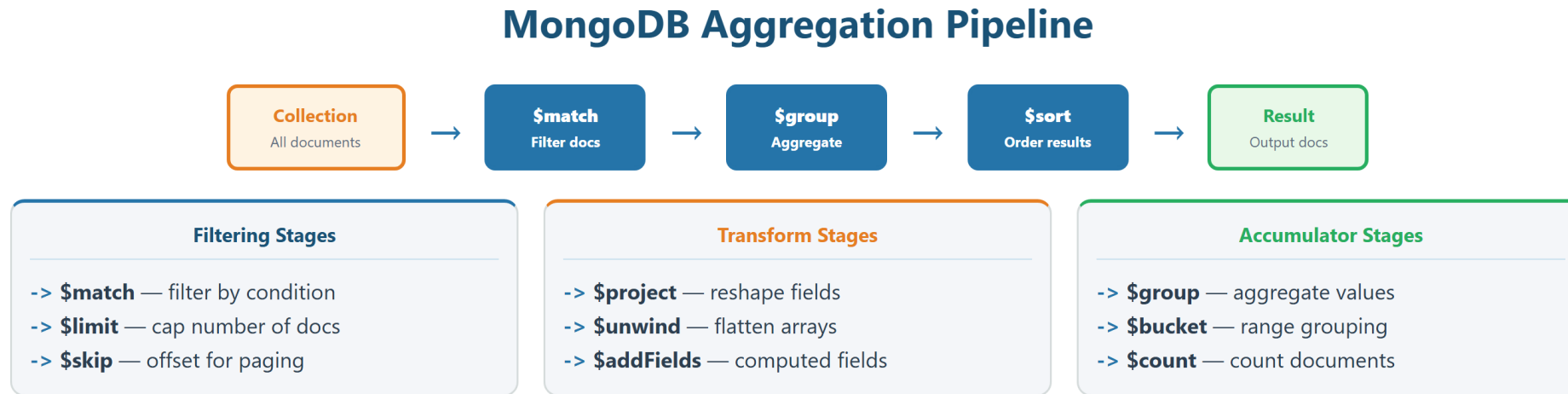
```
// Create a single-field index
db.users.createIndex({ email: 1 }, { unique: true })

// Create a compound index (order matters for queries)
db.orders.createIndex({ customerId: 1, orderDate: -1 })

// Text index for basic $text queries
db.articles.createIndex({ title: "text", body: "text" })
// Note: Use MongoDB Search / Atlas Search for richer production search capabilities

// Check if a query uses an index
db.users.find({ email: "alice@test.com" }).explain("executionStats")
// Look for: winningPlan.stage = 'IXSCAN' (good)
//           winningPlan.stage = 'COLLSCAN' (bad – no index used)
```

MongoDB Aggregation Pipeline



Place \$match early to use indexes — each stage passes output to the next

Aggregation Pipeline Example

```
// Business scenario: monthly revenue by product category
db.orders.aggregate([
  // Stage 1: Filter to completed orders (uses index on status)
  { $match: { status: "completed", orderDate: { $gte: ISODate("2024-01-01") } } },

  // Stage 2: Break out line items from each order
  { $unwind: "$items" },

  // Stage 3: Group by category, calculate totals
  { $group: {
    _id: "$items.category",
    totalRevenue: { $sum: { $multiply: ['$items.price', '$items.qty'] } },
    orderCount: { $sum: 1 }
  } },

  // Stage 4: Sort by revenue descending
  { $sort: { totalRevenue: -1 } }
])
```

Data Modeling Patterns

One-to-Few: Embed

- Phone numbers, addresses — embed directly in the parent document

One-to-Many: Embed or Reference

- Order line items (bounded) — embed; blog comments (unbounded) — reference
- Beware of growing arrays — MongoDB has a 16 MB document size limit

Many-to-Many: Reference

- Students ↔ Courses — store arrays of IDs on one or both sides
- Use \$lookup (LEFT JOIN equivalent) to retrieve related documents



DynamoDB Core Concepts

Section 4



DynamoDB Fundamentals

Fully Managed Service

- AWS provisions and manages all infrastructure — no servers to patch
- Supports both key-value and document data models (JSON-like attributes)

Tables, Items, and Attributes

- Table = collection of items; Item = group of attributes (like a document)
- No fixed schema — each item can have different attributes

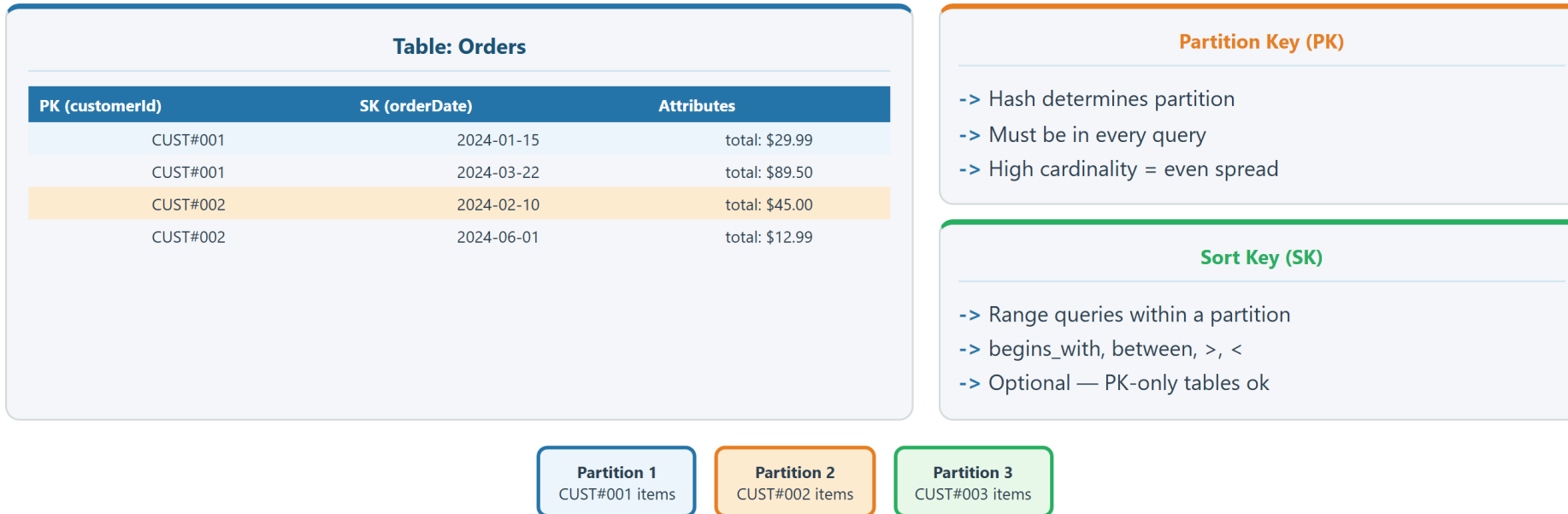
Capacity Modes

- On-Demand — pay per request, auto-scales, ideal for unpredictable workloads
- Provisioned — set read/write capacity units (RCU/WCU), more cost-effective at scale



DynamoDB Partition & Sort Keys

DynamoDB Partition & Sort Key Design



Items with the same partition key are stored together and sorted by sort key



Partition Key Design

Choosing a Good Partition Key

- High cardinality — many distinct values spread items across partitions evenly
- Avoid hot partitions — one busy key overwhelms a single partition

Good Keys vs. Bad Keys

- Good: userId, orderId, deviceId — many unique values
- Bad: status ("active"/"inactive"), date (all today's traffic hits one partition)

Composite Keys Enable Range Queries

- PK = customerId, SK = orderDate — query all orders for a customer in a date range
- SK supports begins_with, between, >, < operators



Query vs. Scan

Query Operation

- Requires partition key — finds items in one partition, very efficient
- Optional sort key condition narrows results within the partition
- Returns items sorted by sort key automatically

Scan Operation

- Reads every item in the table — expensive and slow at scale
- Can apply filters, but only after reading items (still consumes full RCU)
- Avoid scans in production — use indexes instead



DynamoDB Query Examples

```
import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import { QueryCommand, DynamoDBDocumentClient } from '@aws-sdk/lib-dynamodb';

const client = DynamoDBDocumentClient.from(new DynamoDBClient({}));

// Query: Get all orders for a customer in 2024
const result = await client.send(new QueryCommand({
  TableName: "Orders",
  KeyConditionExpression: 'customerId = :pk AND orderDate BETWEEN :start AND :end',
  ExpressionAttributeValues: {
    ':pk': 'CUST#001',
    ':start': '2024-01-01',
    ':end': '2024-12-31',
  },
}));

console.log(result.Items); // sorted by orderDate within CUST#001
```



Read Consistency Models

Eventually Consistent Reads (Default)

- May not reflect the very latest write — typically consistent within milliseconds
- Costs 0.5 RCU per 4 KB item — cheaper and higher throughput

Strongly Consistent Reads

- Always returns the most recent write — guaranteed up-to-date
- Costs 1.0 RCU per 4 KB item — use when correctness is critical

MongoDB Comparison

- Reads from Primary are always consistent; Secondary reads may lag
- writeConcern and readConcern settings control consistency guarantees



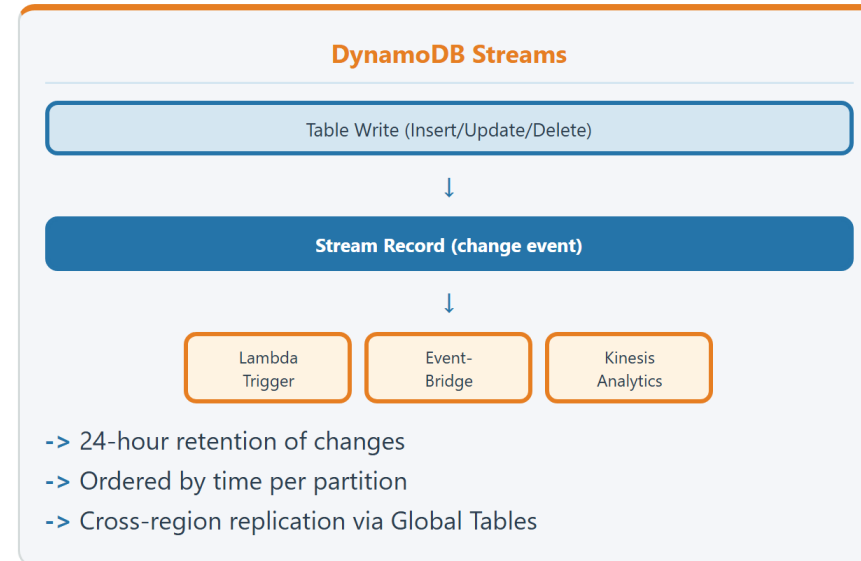
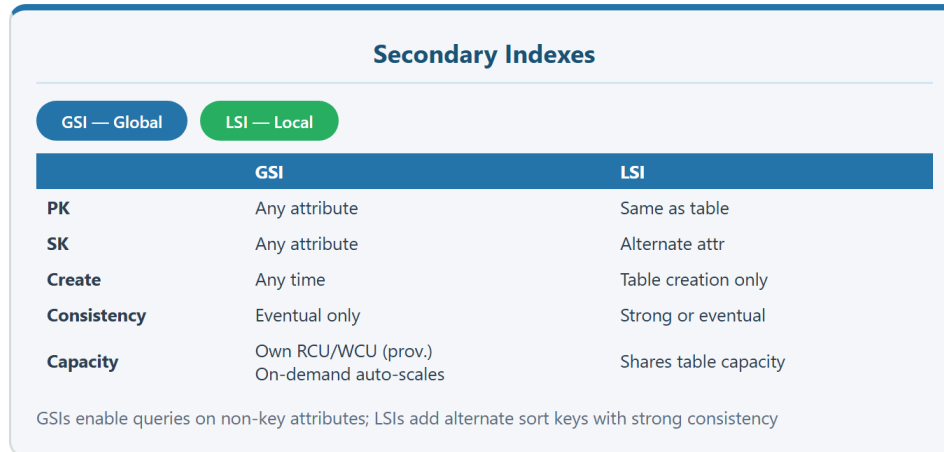
Secondary Indexes and DynamoDB Streams

Section 5



DynamoDB Indexes & Streams

DynamoDB Indexes & Streams



GSI's add query flexibility; LSI's add sort-key options with strong consistency; Streams enable event-driven architectures

Global Secondary Indexes (GSI)

What GSIs Provide

- Query the table using a completely different partition key and sort key
- Can be created or deleted at any time after table creation

GSI Design

- Choose a GSI PK that matches your alternate access pattern
- Project only the attributes you need to minimize storage and cost

Limitations

- Eventually consistent reads only — strongly consistent reads are not supported on GSIs
- On provisioned tables, GSIs have their own read/write capacity settings
- On on-demand tables, GSIs scale in on-demand mode and follow on-demand quotas



Local Secondary Indexes (LSI)

What LSIs Provide

- Same partition key as the base table with an alternate sort key
- Must be defined at table creation time — cannot be added later

LSI vs. GSI

- GSI — different partition key allowed; eventual consistency only
- LSI — same partition key as base table; alternate sort key; supports strong or eventual consistency

When to Use LSIs

- When you need strongly consistent reads on an alternate sort key
- Limit of 5 LSIs per table; share the partition's 10 GB item collection limit



Creating a Global Secondary Index

```
// AWS CLI: Add a GSI to query orders by status + date
aws dynamodb update-table \
  --table-name Orders \
  --attribute-definitions \
    AttributeName=status,AttributeType=S \
    AttributeName=orderDate,AttributeType=S \
  --global-secondary-index-updates '[{
    "Create": {
      "IndexName": "status-date-index",
      "KeySchema": [
        { "AttributeName": "status", "KeyType": "HASH" },
        { "AttributeName": "orderDate", "KeyType": "RANGE" }
      ],
      "Projection": { "ProjectionType": "ALL" },
      "ProvisionedThroughput": { "ReadCapacityUnits": 5, "WriteCapacityUnits": 5 }
    }
  }]'
```



DynamoDB Streams

Change Data Capture

- Streams record every insert, update, and delete as ordered change events
- Retained for 24 hours — consumers can replay if needed

Common Use Cases

- Trigger a Lambda function to send notifications on new orders
- Replicate data to Amazon OpenSearch Service for full-text search and analytics
- Build audit logs and analytics pipelines

Global Tables

- Multi-region, active-active replication powered by DynamoDB Streams



DynamoDB Stream with Lambda Trigger

```
// Lambda handler triggered by DynamoDB Stream events
export const handler = async (event) => {
  for (const record of event.Records) {
    console.log('Event:', record.eventName); // INSERT | MODIFY | REMOVE

    if (record.eventName === 'INSERT') {
      const newItem = record.dynamodb.NewImage;
      // Send welcome email for new user
      await sendNotification({
        email: newItem.email.S,
        message: `Welcome, ${newItem.name.S}!`,
      });
    }
  }
};
```



Choosing the Right Database

Section 6



MongoDB vs. DynamoDB Decision Framework

Choose MongoDB When

- You need rich ad-hoc queries, aggregation pipelines, and flexible exploration
- Your data model is complex with nested documents and varied access patterns
- You want full control over infrastructure (self-hosted) or use Atlas (managed)

Choose DynamoDB When

- You need single-digit millisecond latency at any scale with zero ops
- Your access patterns are well-defined and key-based
- You are in the AWS ecosystem and need tight Lambda/EventBridge integration



Feature Comparison

Query Flexibility

- MongoDB: rich query language, aggregation pipeline, geospatial, and MongoDB Search / Atlas Search capabilities
- DynamoDB: primary-key queries + GSIs; Scan is expensive

Scaling Model

- MongoDB: shard-key design and sharded-cluster operations; balancing and chunk movement are automated
- DynamoDB: automatic partitioning, on-demand scaling

Transactions

- MongoDB: multi-document ACID transactions (since v4.0)
- DynamoDB: TransactWriteItems/TransactGetItems (up to 100 items)



Security Best Practices

MongoDB Security

- Enable authentication (SCRAM-SHA-256) — never run without auth in production
- Use TLS for all client and inter-node connections
- Enable encryption at rest: Atlas default encryption, WiredTiger for self-managed, or disk/KMS-based encryption
- Role-Based Access Control (RBAC) — grant least-privilege roles

DynamoDB Security

- IAM policies control access — attach least-privilege policies to roles
- Encryption at rest (AWS-managed or customer-managed keys) enabled by default
- VPC endpoints keep traffic off the public internet



Operational Considerations

Backup and Recovery

- MongoDB: Atlas continuous backup/cloud snapshots for managed; filesystem snapshots for self-managed; mongodump/mongorestore for logical backup and migration
- DynamoDB: on-demand backups and point-in-time recovery (PITR) up to 35 days

Time-to-Live (TTL)

- Both support automatic document/item expiration for session data, caches, logs

Monitoring

- MongoDB: profiler, slow query log, Atlas performance advisor
- DynamoDB: CloudWatch metrics, contributor insights for hot partitions



Key Takeaways

Key Takeaways

1. NoSQL databases trade rigid schemas and JOINS for flexibility and horizontal scale
2. The CAP theorem means distributed systems choose between consistency and availability under partition
3. MongoDB uses replica sets for HA and sharding for scale — indexes are critical for performance
4. DynamoDB is fully managed — design around partition keys and use GSIs for alternate access patterns
5. DynamoDB Streams enable event-driven architectures with Lambda triggers
6. Choose your database based on access patterns, query needs, and operational requirements

